

```

Join                                0
Index range scans                   0
Join without check                  0
Join with check                     0
Rows sorted                         0
Range sorts                         0
Merge passes                        0
Table scans                        0
Potential filesorts                 0

```

You will see that there is a lot of information that will help you to fix the query. In the simple example given above, you see a really high *Optimizer cost*. In our case, by making the necessary changes to use the right index, and by further narrowing the search results, we were able to optimise the query, and thus bring down the *Optimizer cost*.

Fix them to work under stress

The behaviour of queries can change with load. There are several system load-testing tools available in the market, but in our case we need to load-test only the database. MySQL version 5.1.4 and above comes with *mysqlslap*, a utility that's very useful for load testing. Its advantages are that it is very simple to use, and can be used to connect to and monitor a remote host running on any OS, or a MySQL instance that may not include *mysqlslap*. It provides options to create multiple client threads to load-test the server with a query, and it can iterate several times. It can also read an input file containing queries, and test each of them with the specified user load. Using shell scripts, you can also run it in multiple parallel processes. The important command-line options for load testing are:

- query* (or —*q*): the query to be tested, or a file name if it is in a file.
- concurrency* (or —*c*): the number of client threads to be created. The upper limit is the *max_connections* variable set in the server's *my.cnf* file.
- iterations* (or —*i*): the number of test runs to be executed by each client thread.
- create-schema*: a schema is created for its work, on the machine that is being monitored.

For a more detailed command-line argument reference, please see [4]. Here, we demonstrate load-testing two queries that are alternatives for a given objective. We test each of the queries with load levels of 5, 20, 50, 75, 100 and 200. The command below describes the load testing of *query1*, stored in the file *query1.txt*, with 50 client threads, and 100 iterations. The schema test is created in the local machine.

```

mysqlslap --concurrency=50 --iterations=100 --query=/home/myftp/
query1.txt --create-schema=test
Benchmark

```

Average number of seconds to run all queries: 0.257

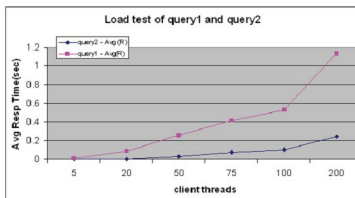


Figure 1: Graph of results

seconds

Minimum number of seconds to run all queries: 0.044

seconds

Maximum number of seconds to run all queries: 0.299

seconds

Number of clients running queries: 50

Average number of queries per client: 1

You can see from the results, and in the graph in Figure 1, that *query2* is more scalable when compared to *query1*.

Client threads	Query 1 response time			Query 2 response time		
	Avg	Min	Max	Avg	Min	Max
5	0.003	0.003	0.005	0	0	0.001
20	0.083	0.012	0.116	0.002	0.002	0.003
50	0.257	0.044	0.299	0.031	0.005	0.074
75	0.414	0.179	0.543	0.071	0.012	0.108
100	0.527	0.387	0.567	0.1	0.015	0.14
200	1.134	1.074	1.176	0.244	0.029	0.292



References

- [1] mysqldumpslow, Summarize Slow Query Log Files, (<http://dev.mysql.com/doc/refman/5.5/en/mysqldumpslow.html>)
- [2] MySQL Administrator's Bible By: Sheeri K. Cabral; Keith Murphy, Publisher: John Wiley & Sons, Pub. Date: May 11, 2009, Print ISBN: 978-0-470-41691-4
- [3] mk-query-profiler, <http://www.maatkit.org/doc/mk-query-profiler.html>
- [4] mysqlslap, <http://dev.mysql.com/doc/refman/5.1/en/mysqlslap.html>

By: Rajeshwari Ganesan and Geetika Goel

Rajeshwari Ganesan is the co-founder and partner of a boutique firm specialising in software performance engineering (www.pea-online.com).

Geetika Goel is a Senior Project Manager at Infosys. She specializes in Technologies catering to Rich User Experience.